Year: 2026
Track: Generative AI Track

Smart Agriculture Advisor for Lebanese Farmers

Joud Senan

March 30, 2026

Contents

1 Introduction **5**

2 Project Goal and Problem Framing **5**

3 Rubric-to-Implementation Mapping **5**

3.1 Two Independent Agent Systems on Different Tech Stacks 5

3.2 RAG Pipeline 5

3.3 MCP Server 6

3.4 Dockerized Microservice Architecture 6

3.5 API Layer 6

3.6 Guardrails and Safety 6

3.7 Evaluation 7

4 System Architecture **7**

4.1 High-Level Architecture 7

4.2 Why a Distributed Architecture Was Chosen 7

5 Agent System A: LangGraph Supervisor and Specialists **7**

5.1 Role of Agent System A 7

5.2 Graph Structure 7

5.3 Why LangGraph Was Appropriate 8

5.4 Deterministic Workflow with Optional ReAct Pattern 8

6 Agent System B: Independent Specialist Service **8**

6.1 Purpose of Agent System B 8

6.2 Execution Modes 8

6.3 Why Agent B Was Kept Separate 8

7 RAG Pipeline Design and Justification **8**

7.1 Why RAG Was Necessary 8

7.2 Document Corpus 9

7.3 Chunking Strategy 9

7.4 Embedding Strategy 9

7.5 Why Qdrant Was Chosen 9

7.6 Retrieval and Reranking . . . 9
7.7 Grounded Prompting . . . 10

8 MCP Server Design **10**
8.1 Purpose of MCP in This Project . . . 10
8.2 Exposed Tools . . . 10
8.3 Why MCP Was Worth Including . . . 10

9 Docker, Networks, and Volumes **10**
9.1 Container Layout . . . 10
9.2 Network Design . . . 10
9.3 Why Volumes Were Used . . . 11

10 API Layer and Session Management **11**
10.1 FastAPI Endpoints . . . 11
10.2 Session Persistence . . . 11
10.3 Timeouts and Error Handling . . . 11

11 Guardrails and Production Thinking **11**
11.1 Input Guardrail . . . 11
11.2 Output Guardrail . . . 11
11.3 Iteration Limits . . . 11
11.4 Graceful Degradation . . . 12

12 Evaluation Methodology **12**
12.1 Test Set . . . 12
12.2 Retrieval Evaluation . . . 12
12.3 Generation Evaluation . . . 12
12.4 Configuration Comparison . . . 12
12.5 How Results Should Be Reported . . . 13

13 Documented Failure Cases and Root-Cause Analysis **13**
13.1 Failure Case 1: Olive Irrigation Query . . . 13
13.2 Failure Case 2: Early Blight Query Routed Too Broadly . . . 13
13.3 Failure Case 3: Abstract Harvest-versus-Hold Query . . . 14

14 Current Limitations **14**

15 Conclusion **14**

1 Introduction

This project implements a production-style smart agriculture assistant designed for farmers, agronomy students, and extension-support settings in Lebanon and the Mediterranean region. The system answers questions about crop management, pests and diseases, irrigation planning, soil interpretation, fertilizer direction, and harvest-versus-hold market timing. Rather than building a single chatbot, the project was intentionally designed as a distributed AI system composed of independent services that communicate over the network.

The objective of the project was not only to answer agricultural questions, but also to satisfy the full final-project requirements of the Generative AI Track. The final system therefore integrates retrieval-augmented generation (RAG), multi-agent orchestration, a secondary independent agent service, an MCP server, containerized deployment, evaluation scripts, session persistence, streaming API support, and production-minded guardrails.

2 Project Goal and Problem Framing

Agricultural decision-making is highly contextual. A useful assistant must go beyond generic language generation and provide grounded answers tied to crop stage, region, soil condition, symptom description, and operational constraints such as storage or perishability. A purely generative model can produce fluent responses, but it may hallucinate or fail to distinguish between similar agronomic situations.

For that reason, the project adopts a retrieval-augmented and service-oriented architecture. The system first retrieves relevant domain evidence from curated agricultural documents, then routes the query to the appropriate specialists, then optionally calls supporting services for irrigation or soil-analysis logic, and finally synthesizes a grounded response. This makes the assistant more explainable, more modular, and more defensible during evaluation.

3 Rubric-to-Implementation Mapping

This section explains how the final system addresses each mandatory requirement.

3.1 Two Independent Agent Systems on Different Tech Stacks

The architecture contains two separate agent systems:

- **Agent System A**: a LangGraph-based orchestration service written in Python and exposed through FastAPI. It is the primary reasoning and routing layer.
- **Agent System B**: an independent FastAPI microservice dedicated to irrigation and weather-style planning. It includes a Google ADK execution path and a deterministic fallback path for reliability.

These two systems are intentionally deployed as separate containers. Agent System A does not import Agent System B as a Python module. Instead, it communicates with it through HTTP requests, which satisfies the requirement for real service boundaries and inter-system communication.

3.2 RAG Pipeline

At least one agent system had to include a full RAG pipeline. This requirement is fulfilled by Agent System A. The pipeline includes:

1. document ingestion from a domain-specific agriculture corpus,
2. chunking using recursive, heading-aware splitting,

3. embedding generation,
4. vector storage in Qdrant,
5. retrieval with metadata-aware filtering,
6. lightweight reranking,
7. grounded answer synthesis.

The indexed corpus spans crop guides, pest and disease references, soil and fertilizer materials, and market planning rules. This comfortably exceeds the minimum content depth required for the project.

3.3 MCP Server

The system includes a dedicated MCP server running in its own container. It exposes multiple agriculture-domain tools, including:

— soil interpretation,
— fertilizer calculation guidance,
— water-use estimation,
— bundled analysis combining multiple soil-related checks.

Agent System A can call the MCP service. It first attempts MCP transport through the streamable HTTP path, and if that fails in a constrained environment, it falls back to the REST bridge hosted by the same MCP container. This preserves both architectural correctness and demo robustness.

3.4 Dockerized Microservice Architecture

The full project runs as a containerized microservice architecture orchestrated using `docker-compose`. The main services are:

— `agent-system-a`
— `agent-system-b`
— `mcp-server`
— `vector-db` (Qdrant)
— `frontend`

Each service has its own Dockerfile and is independently deployable. This makes the submission much closer to production deployment than a single-process notebook-style architecture.

3.5 API Layer

Both agent systems are exposed through FastAPI. Agent System A provides the main `/chat` endpoint, a streaming endpoint, session persistence, and error handling with timeouts and graceful degradation. Agent System B also exposes a chat endpoint and its own health endpoint. Session data for both services is persisted using SQLite-based storage.

3.6 Guardrails and Safety

Production readiness is not only about model quality. It also requires safe handling of invalid or risky requests. The project therefore includes:

— input scope filtering,
— output guardrails and disclaimer enforcement,
— request timeouts,
— iteration limits on agent loops,
— graceful degradation when Agent B or MCP is unavailable.

3.7 Evaluation

The project includes a structured evaluation package:

- a 20-question test set with expected routes, grounded answers, and expected sources,
- retrieval evaluation scripts computing Precision@K, Recall@K, and MRR,
- generation evaluation scripts scoring faithfulness, correctness, and relevance,
- a comparison script for at least two configurations,
- three documented failure cases with root-cause analysis.

4 System Architecture

4.1 High-Level Architecture

The system follows the flow:

$$\text{Frontend} \rightarrow \text{Agent A} \rightarrow (\text{Qdrant} + \text{Agent B} + \text{MCP})$$

The frontend provides the user interface. Agent System A acts as the main orchestrator. Qdrant provides document retrieval. Agent System B provides irrigation and planning support. The MCP server exposes structured domain tools for soil, fertilizer, and water-use logic.

4.2 Why a Distributed Architecture Was Chosen

A single application could have been written to do everything, but that would have hidden important engineering boundaries. The distributed design was chosen for the following reasons:

- to demonstrate real inter-service communication,
- to separate responsibilities across specialist services,
- to make the system easier to explain and maintain,
- to align with production-style deployment,
- to satisfy the rubric requirement that independent systems communicate over the network.

5 Agent System A: LangGraph Supervisor and Specialists

5.1 Role of Agent System A

Agent System A is the primary business-logic layer and the main user-facing reasoning system. It receives the query, parses agricultural entities from it, decides which specialist routes are needed, retrieves evidence from the corpus, calls supporting services when necessary, and synthesizes the final response.

5.2 Graph Structure

The LangGraph workflow includes the following main nodes:

- **Supervisor**: extracts entities and determines routes.
- **Crop Specialist**: retrieves crop-management guidance.
- **Pest Specialist**: retrieves disease, symptom, and IPM guidance.
- **Market Specialist**: retrieves harvest-versus-hold and seasonal market logic.
- **Irrigation Bridge**: calls Agent System B via HTTP.
- **Soil Bridge**: calls the MCP server.
- **Synthesizer**: merges the evidence and generates the final grounded answer.

5.3 Why LangGraph Was Appropriate

LangGraph was selected because the system needed explicit control flow rather than a single large prompt. The project requires routing, conditional specialist execution, loop limits, state passing, and external service calls. LangGraph makes this architecture readable and structured.

5.4 Deterministic Workflow with Optional ReAct Pattern

The packaged submission keeps a deterministic routed graph as the stable default behavior. When a working LLM backend is available, the system can also attempt a ReAct-style pattern using tools such as internal retrieval and web search. If that richer path fails, the service falls back to the deterministic graph instead of failing the whole request. This is an intentional design choice that prioritizes reliability.

6 Agent System B: Independent Specialist Service

6.1 Purpose of Agent System B

Agent System B was designed as a separate specialist service focused on irrigation and weather-style planning. It exists to satisfy the requirement that the project include a second independent agent system with its own logic and runtime path.

6.2 Execution Modes

Agent System B supports two modes:

- **Google ADK mode**: used when the ADK path and required configuration are available.
- **Deterministic fallback mode**: used when the ADK path is unavailable or fails at runtime.

The fallback is not a weakness. It is a deliberate engineering strategy that preserves service continuity and allows the project to run even in low-resource or API-restricted environments.

6.3 Why Agent B Was Kept Separate

Keeping Agent B separate has several advantages:

- it demonstrates real microservice boundaries,
- it isolates irrigation-specific logic from the main RAG workflow,
- it makes testing and debugging easier,
- it improves maintainability and future extensibility.

7 RAG Pipeline Design and Justification

7.1 Why RAG Was Necessary

The assistant answers questions that must remain grounded in agriculture documents rather than in the hidden memory of a model. RAG was therefore necessary to reduce hallucination, support source attribution, and make the assistant easier to trust.

7.2 Document Corpus

The corpus includes several categories of domain knowledge:

- crop production guides,
- pest and disease references,
- soil and fertilizer interpretation notes,
- market and harvest-timing rules.

This structure was chosen because the target questions are heterogeneous. A single flat dataset would not be enough for reliable routing and filtering.

7.3 Chunking Strategy

Chunking is one of the most important parts of a RAG system. The project uses recursive, heading-aware chunking with overlap. The default values are:

- chunk size: 900
- chunk overlap: 180

This choice was made because the corpus is mostly composed of short, structured guide-style documents. Many relevant facts appear in medium-length sections such as pH guidance, irrigation-sensitive growth stages, and symptom-to-treatment explanations. If chunks are too small, this meaning becomes fragmented. If chunks are too large, retrieval precision becomes weaker. The chosen values strike a practical balance between semantic completeness and retrieval focus.

7.4 Embedding Strategy

The packaged configuration uses a deterministic local embedding provider by default. This was chosen for reproducibility and offline robustness. An externally hosted embedding API may fail or introduce latency and instability. A deterministic local mode makes the project reproducible and testable without depending on external quota or connectivity.

This does not mean the architecture is locked to one embedding strategy. The design remains configurable and can support stronger providers later.

7.5 Why Qdrant Was Chosen

Qdrant was selected because it is a production-friendly vector database with persistence, metadata support, and efficient retrieval behavior. Compared with purely in-memory approaches, it is better aligned with a service-based architecture. Metadata support is especially important because the project benefits from filtering by topic or crop before reranking the results.

7.6 Retrieval and Reranking

The system does not rely only on raw top-k vector similarity. It combines:

- vector retrieval,
- metadata-aware filtering,
- lightweight reranking using semantic and lexical cues.

This matters because the corpus is heterogeneous. For example, a pest question should not be dominated by market documents, and a soil query should not retrieve only crop-history snippets. Metadata filtering and reranking therefore improve the quality of the final context sent to synthesis.

7.7 Grounded Prompting

The answer-generation stage is designed to remain grounded in retrieved evidence. When evidence is weak or incomplete, the system is expected to state uncertainty rather than invent unsupported claims. This is especially important in agricultural advice, where overconfident but wrong recommendations could be misleading.

8 MCP Server Design

8.1 Purpose of MCP in This Project

The MCP server provides structured domain tools that are cleaner to expose as callable capabilities than to hard-code directly inside the main orchestration layer. This keeps the system modular and closer to the tool-usage paradigm emphasized during the course.

8.2 Exposed Tools

The main MCP tools are:

- soil analysis,
- fertilizer estimation,
- water-use estimation,
- bundled agronomic analysis.

These tools are relevant because they complement retrieval. Some user questions are not only about looking up a fact; they also require structured interpretation or calculation.

8.3 Why MCP Was Worth Including

The MCP layer demonstrates that the system is more than a retriever-plus-generator. It can expose domain capabilities in a formal tool layer that can be invoked when needed. This is valuable both architecturally and pedagogically because it reflects modern agent-tool integration patterns.

9 Docker, Networks, and Volumes

9.1 Container Layout

Each component is isolated in its own container. This improves modularity and matches the submission requirement that each service be independently deployable.

9.2 Network Design

Two Docker networks are used:

- **public-net**: for the frontend and Agent A,
- **internal-net**: for Agent A, Agent B, the MCP server, and Qdrant.

This design separates the user-facing entry point from internal-only services. It communicates a more production-minded security model than exposing every component directly.

9.3 Why Volumes Were Used

The project uses Docker volumes through bind-mounted runtime directories. This was done so that persistent state survives container restarts and can be inspected directly from the project folder. In practice, the volumes preserve:

- Agent A session history,
- Agent B session history,
- MCP runtime data,
- Qdrant vector storage.

Read-only mounts are also used for datasets, scripts, and evaluation material, which is both safer and cleaner.

10 API Layer and Session Management

10.1 FastAPI Endpoints

The system exposes FastAPI endpoints for chat and health checking. Agent System A also exposes a streaming endpoint, which satisfies the rubric requirement for streaming support.

10.2 Session Persistence

Conversation continuity is important in real assistant systems. The project therefore stores session history in SQLite-backed storage. This allows follow-up questions to use previous context such as crop type, region, soil condition, or earlier clarifications.

10.3 Timeouts and Error Handling

Requests to internal services use timeouts and try-except handling so that a partial failure does not collapse the entire system. Instead, the assistant degrades gracefully and explains when a specialist service was unavailable.

11 Guardrails and Production Thinking

11.1 Input Guardrail

The assistant performs scope filtering so that clearly unrelated requests are rejected early. This is important because the project is domain-specific and should not pretend to answer arbitrary topics with the same level of confidence.

11.2 Output Guardrail

The output layer enforces safer phrasing and grounded behavior. When evidence is incomplete, the assistant should say so. It should not present uncertainty as fact.

11.3 Iteration Limits

The system limits specialist-routing loops and tool-use steps. This is both a safety feature and a cost-control feature, and it prevents unstable recursive behavior.

11.4 Graceful Degradation

One of the strongest production-minded aspects of the system is that it degrades gracefully. If the ADK path is unavailable, Agent B still returns a useful deterministic answer. If the MCP transport is unavailable, the MCP bridge can still fall back. This behavior is preferable to silent failure or full-service interruption.

12 Evaluation Methodology

12.1 Test Set

The evaluation package includes a 20-question test set. Each question includes:

— the user query,

— the expected route,

— the grounded answer,

— the expected source reference.

This makes the evaluation more rigorous than a collection of ad hoc demo prompts.

12.2 Retrieval Evaluation

Retrieval is evaluated independently using:

— Precision@K,

— Recall@K,

— Mean Reciprocal Rank (MRR).

Evaluating retrieval independently is important because poor retrieval cannot be fixed by answer fluency. If the wrong chunks are retrieved, the final answer will remain weak no matter how strong the language model is.

12.3 Generation Evaluation

The project also includes a generation evaluation script that scores:

— faithfulness,

— correctness,

— relevance.

In the packaged build, this evaluation uses a lightweight judge path suitable for a stable offline submission. The architecture still allows later substitution of an LLM-as-judge if a reliable model backend is available.

12.4 Configuration Comparison

The evaluation package includes at least two configuration profiles. This comparison is important because it demonstrates engineering judgment rather than one-shot coding. A strong project does not merely choose a chunk size or top-k value arbitrarily. It tests alternatives and measures the consequences.

Configuration	Retrieval Metrics	Generation Metrics	Interpretation
Baseline localdet top-5, 900/180	P@5 = 0.15 R@5 = 0.60 MRR = 0.4517	Faithfulness = 0.55 Correctness = 0.431 Relevance = 0.4667	Functional but noisy retrieval. The system often finds at least one relevant source, but ranking quality remains weak and answer quality is uneven.
Baseline localdet top-3, 700/120	P@3 = 0.2166 R@3 = 0.65 MRR = 0.5092	Faithfulness = 0.55 Correctness = 0.502 Relevance = 0.5164	Best-performing configuration. Smaller chunks and lower top-k reduced retrieval noise and improved overall answer quality.

Table 1 – Configuration comparison results for retrieval and generation evaluation.

12.5 How Results Should Be Reported

After running the evaluation scripts, the final report should present the measured results in a compact comparison table, as shown in Table 1.

These values were obtained after running:

```
docker compose exec agent-system-a python /app/scripts/evaluate_retrieval.py
docker compose exec agent-system-a python /app/scripts/evaluate_generation.py
docker compose exec agent-system-a python /app/scripts/compare_configs.py
```

The comparison shows that the second configuration, using top-k = 3, chunk size 700, and chunk overlap 120, performed better overall than the baseline configuration. In particular, it improved retrieval precision, retrieval recall, mean reciprocal rank, generation correctness, and generation relevance, while faithfulness remained unchanged. This made it the preferred configuration for the final reported system.

13 Documented Failure Cases and Root-Cause Analysis

13.1 Failure Case 1: Olive Irrigation Query

A representative failure case is the query *"For olives, when is irrigation especially important for high yield?"* The expected source for this question was the olive crop guide, but retrieval failed to return that source among the top results. In the evaluation, this case produced zero precision, zero recall, and zero reciprocal rank.

The root cause appears to be weak alignment between the query wording and the indexed olive-specific irrigation content. The irrigation phrasing biased retrieval toward more general irrigation and soil-related documents instead of the correct olive guide. As a result, the downstream answer lacked strong grounding in the intended source.

A suitable improvement would be to strengthen crop-specific irrigation retrieval, improve metadata-aware filtering, and refine routing so that olive irrigation queries are linked more reliably to olive crop material.

13.2 Failure Case 2: Early Blight Query Routed Too Broadly

Another failure case is the query *"How can early blight pressure be reduced in tomato?"* This question should primarily activate the pest route and retrieve the tomato disease guide. However, the system showed weak retrieval performance and drifted toward broader tomato crop handling.

The root cause is that the system appears to overweight the general crop term *tomato* while underweighting the disease-specific term *early blight*. This causes retrieval and routing to favor crop-level material instead of pest or disease evidence. In evaluation, this case also scored zero on the main retrieval metrics.

The correct system improvement would be to strengthen disease-aware routing, increase the importance of pest terminology during specialist selection, and improve retrieval ranking for disease-related questions.

13.3 Failure Case 3: Abstract Harvest-versus-Hold Query

A third failure case is the query *"What kind of answer should the assistant give for harvest-versus-hold decisions?"* Although this is conceptually a market question, retrieval failed to return the intended market decision-support document among the top results.

The root cause is that the question is abstract and meta-level rather than phrased as a direct farming decision. The indexed market rules are better matched to concrete questions about perishability, storage, and timing than to abstract wording about the *kind* of answer the assistant should provide. Because of this mismatch, retrieval was distracted by other loosely related documents.

The correct system improvement would be to expand the test set with more abstract phrasings, add paraphrase coverage to the corpus or prompts, and strengthen reranking for market-intent questions that are conceptually relevant but lexically indirect.

14 Current Limitations

Although the project satisfies the core rubric requirements, there are still realistic limitations:

- the default embedding mode prioritizes reproducibility over maximum semantic strength,
- the reranking stage is lightweight rather than cross-encoder based,
- the packaged generation evaluator is heuristic rather than a full API-backed judge,
- Agent B includes a fallback mode because stable external-model access cannot always be guaranteed.

These limitations do not weaken the design. They show that the project was built with practical reliability in mind.

15 Conclusion

This project demonstrates a production-style AI system rather than a single isolated prompt application. It combines LangGraph orchestration, a second independent agent service, a dedicated MCP server, Qdrant-backed retrieval, FastAPI APIs, persistent sessions, Dockerized deployment, evaluation scripts, and graceful degradation strategies.

This architecture shows understanding of both AI capabilities and software-engineering discipline. The system does not only answer agricultural questions; it also demonstrates why external knowledge should remain external, why structured service boundaries matter, why evaluation must be separated from intuition, and why production-ready systems must fail safely rather than fail silently.

Appendix: Reproducibility Commands

Copy `.env.example` to `.env`, then run:

```
docker compose up --build -d
docker compose exec agent-system-a python /app/scripts/ingest_to_qdrant.py
docker compose exec agent-system-a python /app/scripts/evaluate_retrieval.py
docker compose exec agent-system-a python /app/scripts/evaluate_generation.py
docker compose exec agent-system-a python /app/scripts/compare_configs.py
```